# Client-side Scripting

COMP1048: Databases and Interfaces (2025-2026)

Matthew Pike and Yanwen Mao

# Overview

# This Lecture

- Explore the necessity of client-side scripting.
- Learn the fundamentals of JavaScript.
- Apply JavaScript in practical scenarios by creating basic client-side scripts.
- Understand the concept of event-driven programming.

**!** DBI Exam

For the DBI exam, you are not required to write JavaScript code. However, it is important to understand the concepts of client-side scripting and event-driven programming.
If you wish to learn more about JavaScript, there are many resources available online. We recommend the Mozilla Developer Network as a good starting point.

# Recap: What makes a web page?

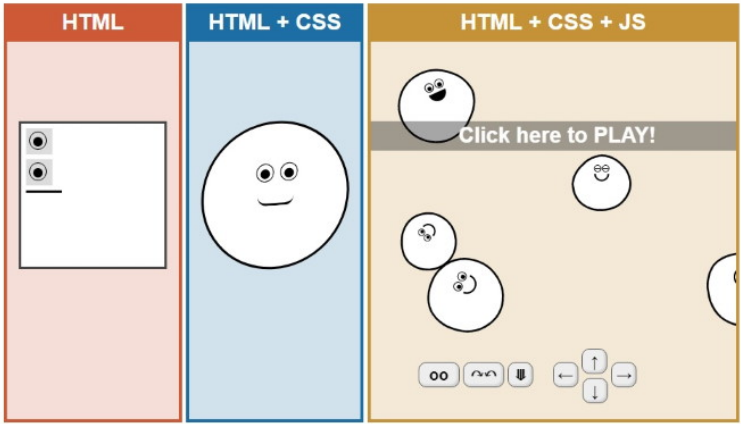



Figure 1: The role of HTML, CSS and JavaScript in a webpage.

# What is Client-side Scripting?

- **Definition**: Execution of scripts in the user's web browser to enhance web page interactivity.
- **Benefits**:
  - Speeds up user interactions.
  - Reduces server load.
  - Allows real-time content updates.
- **Use Cases**:
  - Form validation.
  - Interactive elements (e.g., menus and sliders).
  - Dynamic content loading.
- **Considerations**:
  - Dependent on the user's browser capabilities.
  - Security concerns as code is exposed to the client.
  - Performance concerns as code is executed on the client's machine.
  - Users may disable scripts, meaning execution is not guaranteed.

# Client-side Scripting vs Server-side Scripting

| Aspect | Client-side Scripting | Server-side Scripting |
|---|---|---|
| Execution | In user's web browser | On web server |
| Languages | JavaScript | PHP, Python, Ruby, Java, .NET |
| Use | Enhancing user interface | Backend processes, databases |
| Load on Server | Reduces load | Increases load |
| User Experience | Immediate feedback, dynamic content | May require page reloads |
| Security | Code visible to user | Code hidden from user |

**Table 1:** Client-side Scripting vs Server-side Scripting

# When to Use Client-side Scripting?

## Ideal Scenarios

- **Interactive User Interfaces**: Use for adding responsive elements like drop-down menus and modal pop-ups.
- **Instant Feedback**: Ideal for validating user inputs in forms in real-time.
- **Dynamic Content Updates**: Suitable for updating parts of a web page without a full page reload, often referred to as AJAX (Asynchronous JavaScript and XML).

## Avoid in These Cases

- **Access to Secure Data**: Not suitable for tasks requiring access to server-side databases or sensitive information.
- **Dependence on User's System**: Avoid for critical functions that can be impacted if users have disabled JavaScript.
- **High-Reliability Operations**: Not recommended for processes that must run reliably regardless of the client's browser capabilities.

# JavaScript

# What is JavaScript?

- A lightweight programming language primarily used in web pages.
- Not the same as Java.
- First released by Netscape in 1995.
- Abbreviated as JS.
- Supported by all major web browsers and considered a web standard.

# What can JavaScript do?

- Add interactivity to a web page by reacting to user, page, and state events.
- Update web page content without reloading the page using the Document Object Model (DOM).
- Modify content, style, and attributes of HTML elements.
- Send and receive data from a web server without reloading the page.
- Create cookies to store information about the user on their computer.
- Detect the user's browser and operating system.
- Validate user input before sending it to the server.

# JavaScript: Hello World

- Embedded in HTML with `<script>` tag.
- Included directly in HTML or via external file with `<script src="myScript.js">`.

```
<!DOCTYPE html>
<html>
    <head>
        <title>JavaScript: Hello World</title>
    </head>
    <body>
        <h1>JavaScript: Hello World</h1>
        <script>
            alert("Hello World!");
        </script>
    </body>
</html>
```

# Event-driven Programming

# What is Event-driven Programming?

- A programming paradigm where the flow of the program is determined by events.
- Events can be user actions (e.g., mouse clicks, key presses), sensor outputs, or messages from other programs or threads.
- In JavaScript, event handling typically involves assigning functions to event handlers.

```
# Assign a function to the onclick
# event handler of a button
b = document.getElementById("myButton");

# When the button is clicked, the
# function is executed
b.onclick = function() {
    alert("Hello World!");
}
```

# Event-driven Programming: Example

```
<!DOCTYPE html>
<html>
    <head> <title>Event-driven Programming: Example</title> </head>
    <body>
        <h1>Event-driven Programming: Example</h1>
        <button id="myButton">Click Me!</button>
        <script>
            b = document.getElementById("myButton");
            b.onclick = function() {
                alert("Hello World!");
            }
        </script>
    </body>
</html>
```

# Form Validation with JavaScript

# Form Validation

- JavaScript validates input before submission, enhancing user experience and reducing server load.
- Server-side validation is essential as JavaScript can be disabled.
- Validation methods:
  - `onsubmit` for pre-submission checks.
  - `oninput` for real-time validation.
- Examples:
  - Show/hide elements based on input.
  - Highlight invalid fields.
  - Display submission messages.

# Form Validation: Example (HTML)

- The `onsubmit` event is triggered when the form is submitted.
- `validateForm()` is triggered when the form is submitted.

```
<h1>Form Validation</h1>
<form id="myForm" onsubmit="return validateForm()">
    <input type="text" id="name" placeholder="Name">
    <input type="email" id="email" placeholder="Email">
    <input type="submit" value="Submit">
</form>
```

# Form Validation: Example (JavaScript)

```
function validateForm() {
    var name = document.getElementById("name").value;
    var email = document.getElementById("email").value;
    if (name.length < 3) {
        alert("Name is too short");
        return false;
    }
    if (email.length < 3 || !email.includes("@")) {
        alert("Enter a valid email");
        return false;
    }
    return true;
}
```

# Interacting with the DOM

# Recap: The Document Object Model (DOM)

- DOM is a programming interface for HTML/XML documents.
- Represents documents as nodes and objects.
- Allows JavaScript to dynamically modify structure, style, and content.
- Enables real-time updates without reloading the page.

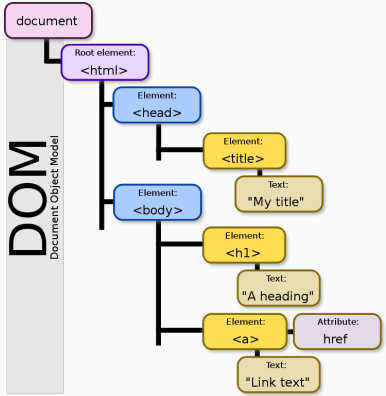



**Figure 2:** A visual representation of the DOM

# DOM Interaction with JavaScript

- JavaScript can modify the DOM to change content, style, or attributes of HTML elements.
- It allows adding new HTML elements and setting event handlers.
- HTML elements can be accessed using the `document` object:
  - `document.getElementById(id)`: Retrieves an element by its id.
  - `document.getElementsByClassName(className)`: Retrieves elements by their class name.
  - `document.getElementsByTagName(tagName)`: Retrieves elements by their tag name.

# DOM Interaction with JavaScript: Example

```
<p id="myParagraph">Original text.</p>
<script>
    // Function to change the text of the paragraph
    function changeText(newText) {
        document.getElementById("myParagraph").textContent = newText;
    }

    // Call changeText function after a delay of 5 seconds
    setTimeout(() => changeText("Updated text after 5 seconds."), 5000);
</script>
```

# Resources

- Mozilla Developer Network
  - https://developer.mozilla.org/en-US/docs/Web/JavaScript
- Learn JavaScript Online
  - https://learnjavascript.online
- https://javascript.info